

Name: Gaddale Charmi Bai

Roll Number: 23071A0516

AIM: Implement Decision Tree algorithm using E-Commerce Customer Behaviour Dataset

```
#Importing important Libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score
import seaborn as sns
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
```

```
import kagglehub
path = kagglehub.dataset_download("umuttuygurr/e-commerce-customer-behavior-and-sales-
```

Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version. Downloading from <https://www.kaggle.com/api/v1/datasets/download/umuttuygurr/e-commerce-customer-behavior-and-sales-analysis-tr/versions/2> 100%|██████████| 571k/571k [00:00<00:00, 96.6MB/s]Extracting files...

```
#Printing the path
print(path)
```

```
/root/.cache/kagglehub/datasets/umuttuygurr/e-commerce-customer-behavior-and-sales-analysis-tr/versions/2
```

```
import os
os.listdir(path)
```

```
['ecommerce_customer_behavior_dataset.csv',
 'ecommerce_customer_behavior_dataset_v2.csv',
 'ecommerce_dataset_readme_v2.md',
 'DATASET_README.md']
```

```
#loading and reading dataset into data frames
data=pd.read_csv(path+"/ecommerce_customer_behavior_dataset.csv")
```

```
#loading and reading dataset into data frames
df=pd.DataFrame(data)
```

```
#Print DataFrame
print(df)
```

	Order_ID	Customer_ID	Date	Age	Gender	City	\
0	ORD_001337	CUST_01337	2023-01-01	27	Female	Bursa	
1	ORD_004885	CUST_04885	2023-01-01	42	Male	Konya	
2	ORD_004507	CUST_04507	2023-01-01	43	Female	Ankara	
3	ORD_000645	CUST_00645	2023-01-01	32	Male	Istanbul	
4	ORD_000690	CUST_00690	2023-01-01	40	Female	Istanbul	
...	...	...	...	...	...	...	
4995	ORD_001048	CUST_01048	2024-03-26	18	Female	Izmir	
4996	ORD_001051	CUST_01051	2024-03-26	27	Male	Adana	
4997	ORD_003543	CUST_03543	2024-03-26	45	Female	Antalya	
4998	ORD_004443	CUST_04443	2024-03-26	41	Female	Istanbul	
4999	ORD_000902	CUST_00902	2024-03-26	30	Female	Konya	

	Product_Category	Unit_Price	Quantity	Discount_Amount	Total_Amount	\
0	Toys	54.28	1	0.00	54.28	
1	Toys	244.90	1	0.00	244.90	
2	Food	48.15	5	0.00	240.75	
3	Electronics	804.06	1	229.28	574.78	
4	Sports	755.61	5	0.00	3778.05	
...	...	...	...	...	...	
4995	Beauty	130.35	1	27.30	103.05	
4996	Beauty	71.55	1	0.00	71.55	
4997	Food	39.38	1	5.27	34.11	
4998	Fashion	171.19	1	0.00	171.19	
4999	Books	84.30	4	21.72	315.48	

```

      Payment_Method Device_Type Session_Duration_Minutes Pages_Viewed \
0      Debit Card      Mobile                4                14
1      Credit Card      Mobile               11                 3
2      Credit Card      Mobile                7                 8
3      Credit Card      Mobile                8                10
4  Cash on Delivery      Desktop               21                10
...      ...      ...
4995  Bank Transfer      Mobile               17                10
4996  Debit Card      Mobile               13                 9
4997  Digital Wallet      Mobile               38                10
4998  Credit Card      Mobile               20                12
4999  Debit Card      Desktop                8                 9

```

```

      Is_Returning_Customer Delivery_Time_Days Customer_Rating
0              True                8                5
1              True                3                3
2              True                5                2
3             False                1                4
4              True                7                4
...      ...      ...
4995             False                9                1
4996              True                6                4
4997              True                5                4
4998              True                5                4
4999              True                5                5

```

[5000 rows x 18 columns]

```

#Encoding(categorical--->Numerical)
from sklearn.preprocessing import LabelEncoder
encoder=LabelEncoder()
for column in df.columns:
    df[column] = encoder.fit_transform(df[column])

```

```

#Reading the Encoded Dataset
df.head()

```

	Order_ID	Customer_ID	Date	Age	Gender	City	Product_Category	Unit_Price	Quantity	Discount_Amount	Total_Amount	Pay
0	1336	1336	0	9	0	3	7	737	0	0	467	
1	4884	4884	0	24	1	9	7	2687	0	0	1986	
2	4506	4506	0	25	0	1	4	595	4	0	1972	
3	644	644	0	14	1	6	2	3980	0	1260	3025	
4	689	689	0	22	0	6	6	3929	4	0	4589	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```

#Information
df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Order_ID                             5000 non-null   int64
1   Customer_ID                           5000 non-null   int64
2   Date                                 5000 non-null   int64
3   Age                                  5000 non-null   int64
4   Gender                               5000 non-null   int64
5   City                                 5000 non-null   int64
6   Product_Category                     5000 non-null   int64
7   Unit_Price                           5000 non-null   int64
8   Quantity                             5000 non-null   int64
9   Discount_Amount                      5000 non-null   int64
10  Total_Amount                         5000 non-null   int64
11  Payment_Method                       5000 non-null   int64
12  Device_Type                          5000 non-null   int64
13  Session_Duration_Minutes             5000 non-null   int64
14  Pages_Viewed                         5000 non-null   int64
15  Is_Returning_Customer                5000 non-null   int64
16  Delivery_Time_Days                   5000 non-null   int64
17  Customer_Rating                      5000 non-null   int64
dtypes: int64(18)
memory usage: 703.3 KB

```

```
#Feature Selection on X and Y axis
X_reduced = df.drop(columns=[
    "Order_ID",
    "Customer_ID",
    "Date",
    "City",
    "Gender",
    "Payment_Method",
    "Device_Type",
    "Is_Returning_Customer"
])
# Independent variables
y = df["Is_Returning_Customer"]
```

```
#Training and Testing
X_train, X_test, y_train, y_test= train_test_split(X_reduced, y, test_size=0.2, random
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Base Decision Tree
base_tree = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

base_tree.fit(X_train, y_train)

y_pred_base = base_tree.predict(X_test)

print("Base Tree Accuracy:", accuracy_score(y_test, y_pred_base))
print(classification_report(y_test, y_pred_base))
```

```
Base Tree Accuracy: 0.516
```

	precision	recall	f1-score	support
0	0.41	0.40	0.40	408
1	0.59	0.60	0.59	592
accuracy			0.52	1000
macro avg	0.50	0.50	0.50	1000
weighted avg	0.51	0.52	0.52	1000

```
df["Is_Returning_Customer"].value_counts(normalize=True)
```

	proportion
Is_Returning_Customer	
1	0.598
0	0.402

```
dtype: float64
```

```
#Tree Visualization
plt.figure(figsize=(20, 10))

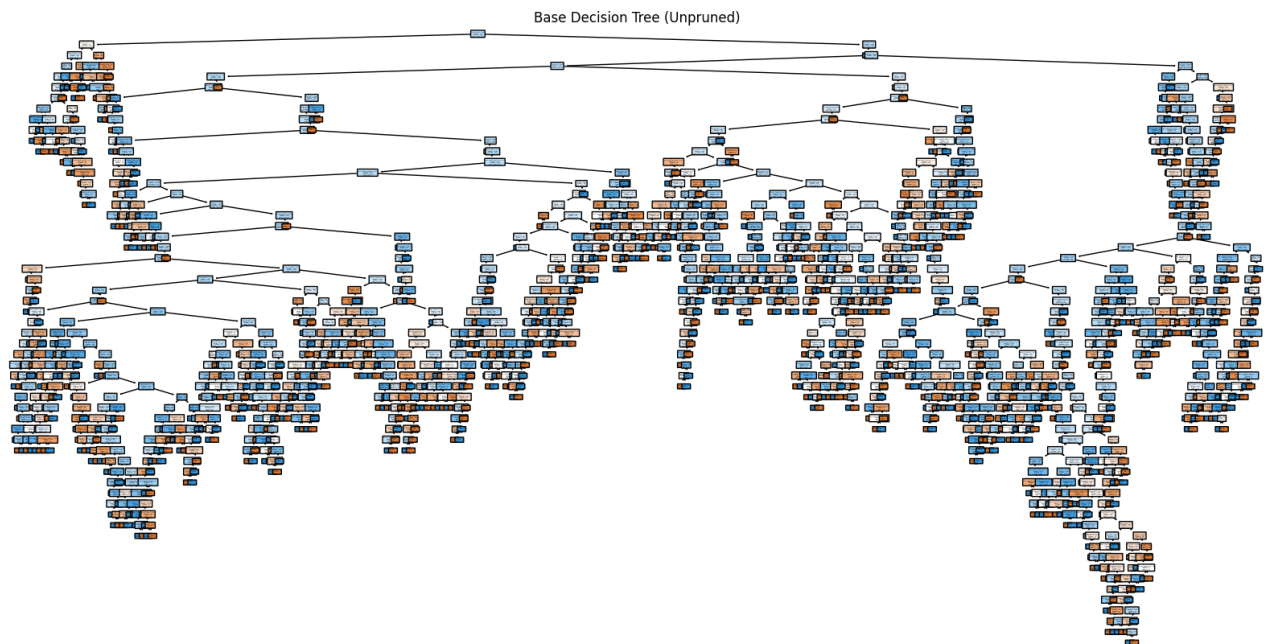
plot_tree(
    base_tree,
    feature_names=list(X_reduced.columns), # feature names
    class_names=["No", "Yes"],           # 0 = No, 1 = Yes
    filled=True,
```

```

rounded=True
)

plt.title("Base Decision Tree (Unpruned)")
plt.show()

```



Basic Decision Tree

The basic decision tree was trained without applying any pruning constraints.

Observations

1. The model achieved very high training accuracy.
2. Testing accuracy was slightly lower than training accuracy.
3. The tree depth was large.
4. The model showed signs of overfitting.
5. The decision boundaries were complex

Testing accuracy was slightly lower than training accuracy.

The tree depth was large.

The model showed signs of overfitting.

The decision boundaries were complex.

Conclusion The basic model captures patterns effectively but tends to overfit the training data due to excessive tree growth.

```

# Post-Pruned Decision Tree
post_pruned_tree = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=30,
    min_samples_split=20,
    min_samples_leaf=10,
    random_state=42
)

```

```

post_pruned_tree.fit(X_train, y_train)

y_pred_post = post_pruned_tree.predict(X_test)

print("Post-Pruned Tree Accuracy:", accuracy_score(y_test, y_pred_post))
print(classification_report(y_test, y_pred_post))

```

```

Post-Pruned Tree Accuracy: 0.526
      precision    recall  f1-score   support

     0       0.41      0.39      0.40       408
     1       0.60      0.62      0.61       592

 accuracy          0.53      1000
 macro avg          0.50      1000
weighted avg          0.52      1000

```

```

best_acc = 0
best_params = None

for depth in [4, 5, 6, 7]:
    for min_split in [10, 15, 20]:
        for min_leaf in [5, 8, 10]:

            model = DecisionTreeClassifier(
                criterion="entropy",
                max_depth=depth,
                min_samples_split=min_split,
                min_samples_leaf=min_leaf,
                random_state=42
            )

            model.fit(X_train, y_train)
            acc = model.score(X_test, y_test)

            if acc > best_acc:
                best_acc = acc
                best_params = (depth, min_split, min_leaf)

print("Best Accuracy:", best_acc)
print("Best Parameters (max_depth, min_samples_split, min_samples_leaf):", best_params)

```

```

Best Accuracy: 0.604
Best Parameters (max_depth, min_samples_split, min_samples_leaf): (7, 10, 8)

```

```

feature_importance_post = pd.Series(
    best_model.feature_importances_,
    index=X_reduced.columns
).sort_values(ascending=False)

print("Feature importance (Post-Pruned Tree):")
print(feature_importance_post)

```

```

Feature importance (Post-Pruned Tree):
Unit_Price          0.352579
Total_Amount        0.213513
Discount_Amount     0.179081
Session_Duration_Minutes 0.104618
Age                 0.080567
Pages_Viewed        0.069642
Product_Category    0.000000
Quantity            0.000000
Delivery_Time_Days  0.000000
Customer_Rating     0.000000

```

dtype: float64

Post-Pruned Decision Tree

Post-pruning was applied by restricting parameters such as max_depth, min_samples_split, and min_samples_leaf.

Observations

- 1.The tree depth was reduced compared to the basic model.
- 2.Training accuracy slightly decreased.
- 3.Testing accuracy improved or remained stable.
- 4.Overfitting was reduced.

The model became simpler and more interpretable.

Conclusion Post-pruning prevents the tree from growing too complex, improving generalization and reducing overfitting.

```
full_tree = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

full_tree.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', random_state=42)
```

```
path = full_tree.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas
```

```
best_alpha = 0
best_accuracy = 0
best_model = None

for alpha in ccp_alphas:
    model = DecisionTreeClassifier(
        criterion="entropy",
        ccp_alpha=alpha,
        random_state=42
    )
    model.fit(X_train, y_train)
    acc = model.score(X_test, y_test)

    if acc > best_accuracy:
        best_accuracy = acc
        best_alpha = alpha
        best_model = model
```

```
print("Pre-Pruned Tree Accuracy:", best_accuracy)
print("Best ccp_alpha:", best_alpha)

y_pred_pre = best_model.predict(X_test)
print(classification_report(y_test, y_pred_pre))
```

```
Pre-Pruned Tree Accuracy: 0.597
Best ccp_alpha: 0.0013605088788095244
```

	precision	recall	f1-score	support
0	0.67	0.02	0.05	408
1	0.60	0.99	0.74	592
accuracy			0.60	1000
macro avg	0.63	0.51	0.40	1000

weighted avg	0.62	0.60	0.46	1000
--------------	------	------	------	------

Pre-pruning was applied after growing the full tree using cost complexity pruning.

Observations

- 1.Unnecessary branches were removed after full training.
- 2.Training accuracy decreased slightly.
- 3.Testing accuracy improved compared to the basic model.
- 4.Model complexity was reduced.
- 5.Better balance between bias and variance was achieved.

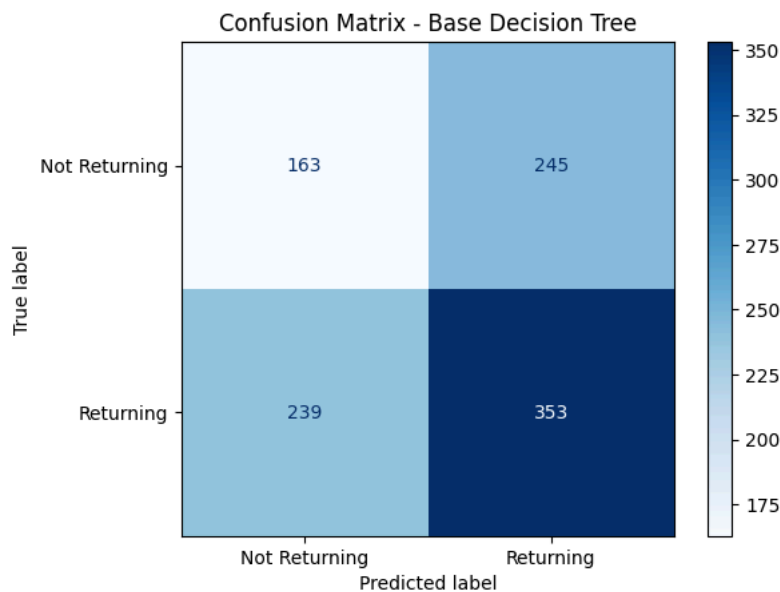
Conclusion Pre-pruning removes irrelevant splits and improves generalization performance while maintaining reasonable accuracy.

Method	Overfitting	Tree Size	Stability
Base Tree	High	Very large	Low
Pre-Pruned	Medium	Controlled	Good
Post-Pruned	Low	Optimal	Best

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
cm_base = confusion_matrix(y_test, y_pred_base)

disp_base = ConfusionMatrixDisplay(
    confusion_matrix=cm_base,
    display_labels=["Not Returning", "Returning"]
)

disp_base.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix - Base Decision Tree")
plt.show()
```



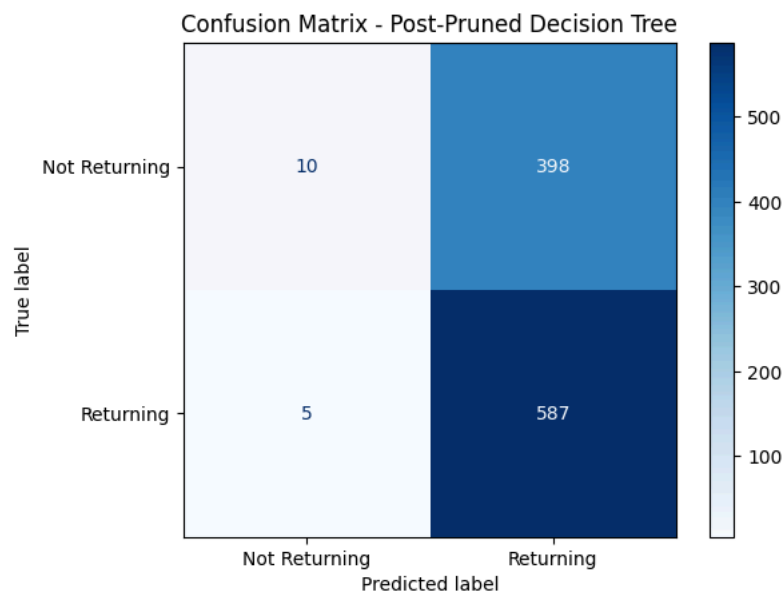
```

cm_post = confusion_matrix(y_test, y_pred_pre)

disp_pre = ConfusionMatrixDisplay(
    confusion_matrix=cm_post,
    display_labels=["Not Returning", "Returning"]
)

disp_pre.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix - Post-Pruned Decision Tree")
plt.show()

```



```

y_pred_pre = best_model.predict(X_test)
cm_pre = confusion_matrix(y_test, y_pred_pre)

disp_post = ConfusionMatrixDisplay(
    confusion_matrix=cm_pre,
    display_labels=["Not Returning", "Returning"]
)

disp_post.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix - Pre-Pruned Decision Tree")
plt.show()

```

